EXP NO: 1 Date: 16/7/2026	SETTING UP THE ENVIRONMENT AND PREPROCESSING THE DATA

AIM:

To set up a fully functional machine learning development environment and to perform data preprocessing operations like handling missing values, encoding categorical variables, feature scaling, and splitting datasets**.**

ALGORITHM:

1. Install Required Libraries:

• Install numpy, pandas, matplotlib, seaborn, and scikit-learn using pip. 2. Import Libraries.

3. Load Dataset:

• Load any dataset (e.g., Titanic or Iris) using pandas.

4. Data Exploration:

• Use df.info(), df.describe(), df.isnull().sum() to understand the data. 5. Handle Missing Values:

• Use .fillna() or .dropna() depending on the strategy.

6. Encode Categorical Data:

• Use pd.get_dummies() or LabelEncoder.

7. Feature Scaling:

• Normalize or standardize the numerical features using StandardScaler or MinMaxScaler.

8. Split Dataset:

• Use train_test_split() from sklearn to create training and testing sets. 9. Display the Preprocessed Data.

CODE:

In [12]:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("ecommerce_sales_analytics_5000.csv")
print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum())
numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())
categorical_cols = df.select_dtypes(include=["object","string"]).columns
for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
df.drop_duplicates(inplace=True)
le = LabelEncoder()
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
scaler = StandardScaler()
numerical_cols = [
    "quantity",
    "unit_price",
    "discount",
    "delivery_days",
    "customer_rating",
    "revenue"
]
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
X = df.drop("revenue", axis=1)
y = df["revenue"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Training Data Shape:", X_train.shape)
print("Test Data Shape:", X_test.shape)
print("\nTraining Data")
print(X_train.head())
```

OUTPUT:

```
   order_id order_date  customer_id product_category region  quantity  \
0     10001   1/1/2022         1102           Beauty  South         7
1     10002   1/2/2022         1435         Clothing  South         7
2     10003   1/3/2022         1860           Beauty   East         3
3     10004   1/4/2022         1270      Electronics   West         5
4     10005   1/5/2022         1106         Clothing   West         5

   unit_price  discount payment_method  delivery_days  customer_rating  \
0      373.65      0.28         Wallet             10              4.7
1       47.74      0.09           Card              6              3.9
2      311.28      0.31            COD              6              2.5
3      524.47      0.02         Wallet              6              1.6
4      139.87      0.33         Wallet              4              4.9

   revenue
0  1883.20
1   304.10
2   644.35
3  2569.90
4   468.56
<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   order_id          5000 non-null   int64
 1   order_date        5000 non-null   str
 2   customer_id       5000 non-null   int64
 3   product_category  5000 non-null   str
 4   region            5000 non-null   str
 5   quantity          5000 non-null   int64
 6   unit_price        5000 non-null   float64
 7   discount          5000 non-null   float64
 8   payment_method    5000 non-null   str
 9   delivery_days     5000 non-null   int64
 10  customer_rating   5000 non-null   float64
 11  revenue           5000 non-null   float64
dtypes: float64(4), int64(4), str(4)
memory usage: 468.9 KB
None
```

None

	order_id	customer_id	quantity	unit_price	discount	\
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	
mean	12500.500000	1505.701200	4.044800	308.418774	0.179984	
std	1443.520003	290.836902	2.020398	169.259369	0.101404	
min	10001.000000	1000.000000	1.000000	15.150000	0.000000	
25%	11250.750000	1253.000000	2.000000	161.895000	0.090000	
50%	12500.500000	1510.000000	4.000000	309.890000	0.180000	
75%	13750.250000	1761.000000	6.000000	455.557500	0.270000	
max	15000.000000	1999.000000	7.000000	599.960000	0.350000	

	delivery_days	customer_rating	revenue
count	5000.000000	5000.000000	5000.000000
mean	6.118800	2.973980	1021.955148
std	3.153264	1.157722	825.584219
min	1.000000	1.000000	11.210000
25%	3.000000	2.000000	354.527500
50%	6.000000	3.000000	796.650000
75%	9.000000	4.000000	1515.690000
max	11.000000	5.000000	4119.330000

```
order_id            0
order_date          0
customer_id         0
product_category    0
region              0
quantity            0
unit_price          0
discount            0
payment_method      0
delivery_days       0
customer_rating     0
revenue             0
dtype: int64
Training Data Shape: (4000, 11)
Test Data Shape: (1000, 11)
```

Training Data

	order_id	order_date	customer_id	product_category	region	quantity
4227	14228	4038	1155	0	2	1.462828
4676	14677	615	1394	2	2	-0.517177
800	10801	2055	1505	3	2	-0.022176
3671	13672	178	1110	0	1	-0.517177
4193	14194	3562	1189	2	2	1.462828

	unit_price	discount	payment_method	delivery_days	customer_rating
4227	1.201833	1.578165	0	-0.989169	1.491025
4676	-1.184684	-0.986097	0	1.548138	-1.273300
800	0.986816	-1.676475	1	1.548138	0.627173
3671	-1.583874	0.197409	2	1.230975	-1.446070
4193	0.844713	-0.492970	2	-1.623496	-0.063908

RESULT:

The Python environment was successfully set up and the dataset was pre-processed by handling missing values, encoding categorical data, performing feature scaling, and splitting the data into training and testing sets. The dataset is now ready for model training and analysis.